voicepls · AI Solutions Agency

API & TECHNICAL DOCUMENTATION

Nobby Restaurant & Lounge — Premium Website

Client: **Taha Khan (Nobby Restaurant & Lounge)**

Prepared by: **Syed Owais Ali Shah**

Project Managed by: **Syed Owais Ali Shah, Aqib Ali Ghumro**

Project Lead: **Abdul Samad Siddiqui**

Date: **August 2026**

Companion to: **Software Design Document (SDD)**

1. Introduction

This document is the technical companion to the Software Design Document (SDD). It gives the development team everything needed to build and run the system: the full API reference, the Docker setup the backend runs in, key frontend implementation notes, and the database schema with example SQL queries.

Architecture recap: Next.js frontend, NestJS API, PostgreSQL database, media served from a CDN — all as defined in the SDD.

2. API Documentation

2.1 Conventions

- Base URL: https://api.nobbyrestaurant.com/api (local development: http://localhost:3000/api)
- All requests and responses use JSON.
- Admin-only endpoints require an Authorization: Bearer <token> header, obtained from the login endpoint.
- Dates and times use ISO 8601 (e.g. 2026-08-10T20:00:00+05:00).
- Prices are in PKR, as plain numbers (e.g. 2800 = Rs. 2,800).

2.2 Endpoint Reference

Full list of endpoints in the system:

Method	Path	Auth	Description
POST	`/auth/login`	Public	Admin login — returns a JWT access token
GET	`/menu-categories`	Public	List all menu categories
POST	`/menu-categories`	Admin	Create a menu category
PATCH	`/menu-categories/:id`	Admin	Update a menu category
DELETE	`/menu-categories/:id`	Admin	Delete a menu category
GET	`/menu-items`	Public	List menu items (optional ?category=id)
GET	`/menu-items/:id`	Public	Get a single menu item
POST	`/menu-items`	Admin	Create a menu item
PATCH	`/menu-items/:id`	Admin	Update a menu item
DELETE	`/menu-items/:id`	Admin	Delete a menu item
GET	`/gallery`	Public	List gallery images
POST	`/gallery`	Admin	Add a gallery image
DELETE	`/gallery/:id`	Admin	Remove a gallery image

GET	/testimonials	Public	List testimonials
POST	/testimonials	Admin	Add a testimonial
DELETE	/testimonials/:id	Admin	Remove a testimonial
POST	/reservations	Public	Submit a reservation request
GET	/reservations	Admin	List reservations (optional ?status=)
PATCH	/reservations/:id/status	Admin	Confirm or cancel a reservation
POST	/contact	Public	Submit a contact message
GET	/contact	Admin	List contact messages

2.3 Authentication

POST /auth/login *[Public]*

Admin login. Returns a JWT used for all Admin-marked endpoints below.

```
Request body
{
  "email": "admin@nobbyrestaurant.com",
  "password": "••••••••"
}

Response  200 OK
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresIn": 3600
}
```

The access token is sent as Authorization: Bearer <accessToken> on every Admin request.

2.4 Menu Categories

GET /menu-categories *[Public]*

```
Response  200 OK
[
  { "id": 1, "name": "Starters", "sortOrder": 1 },
  { "id": 2, "name": "Main Course", "sortOrder": 2 },
  { "id": 3, "name": "Desserts", "sortOrder": 3 }
]
```

POST /menu-categories *[Admin]*

```
Request body
{ "name": "Beverages", "sortOrder": 4 }

Response  201 Created
{ "id": 4, "name": "Beverages", "sortOrder": 4 }
```

PATCH /menu-categories/:id and DELETE /menu-categories/:id follow the same request/response shape — PATCH returns the updated record, DELETE returns 204 No Content.

2.5 Menu Items

GET `/menu-items?category=2` *[Public]*

```
Response  200 OK
[
  {
    "id": 12,
    "categoryId": 2,
    "name": "Grilled Lamb Chops",
    "description": "Char-grilled lamb chops with rosemary jus",
    "price": 2800,
    "imageUrl": "https://cdn.nobbyrestaurant.com/menu/lamb-chops.jpg",
    "isFeatured": true
  }
]
```

POST `/menu-items` *[Admin]*

```
Request body
{
  "categoryId": 2,
  "name": "Grilled Lamb Chops",
  "description": "Char-grilled lamb chops with rosemary jus",
  "price": 2800,
  "imageUrl": "https://cdn.nobbyrestaurant.com/menu/lamb-chops.jpg",
  "isFeatured": true
}

Response  201 Created
{ "id": 12, "categoryId": 2, "name": "Grilled Lamb Chops", ... }
```

PATCH /menu-items/:id and DELETE /menu-items/:id follow the same pattern as above.

2.6 Gallery

GET `/gallery` *[Public]*

```
Response  200 OK
[
  { "id": 5, "imageUrl": "https://cdn.nobbyrestaurant.com/gallery/interior-1.jpg", "caption": "Main
dining hall", "tag": "interior" }
]
```

POST `/gallery` *[Admin]*

```
Request body
{ "imageUrl": "https://cdn.nobbyrestaurant.com/gallery/interior-1.jpg", "caption": "Main dining
hall", "tag": "interior" }

Response  201 Created
```

2.7 Testimonials

GET `/testimonials` *[Public]*

```
Response  200 OK
[
  { "id": 3, "guestName": "Bilal Ahmed", "quote": "Best steak in Karachi, hands down.", "rating": 5
}
]
```

POST `/testimonials` *[Admin]*

```
Request body
{ "guestName": "Bilal Ahmed", "quote": "Best steak in Karachi, hands down.", "rating": 5 }

Response  201 Created
```

2.8 Reservations

POST `/reservations` *[Public]*

Submitted by a visitor from the Reservations page.

```
Request body
{
  "name": "Ali Raza",
  "phone": "+92 300 1234567",
  "partySize": 4,
  "reservationTime": "2026-08-10T20:00:00+05:00"
}

Response  201 Created
{ "id": 45, "status": "pending", "reservationTime": "2026-08-10T20:00:00+05:00" }
```

GET `/reservations?status=pending` *[Admin]*

```
Response  200 OK
[
  { "id": 45, "name": "Ali Raza", "phone": "+92 300 1234567", "partySize": 4,
    "reservationTime": "2026-08-10T20:00:00+05:00", "status": "pending" }
]
```

PATCH `/reservations/45/status` *[Admin]*

```
Request body
{ "status": "confirmed" }

Response  200 OK
{ "id": 45, "status": "confirmed" }
```

2.9 Contact Messages

POST `/contact` *[Public]*

```
Request body
{ "name": "Sara Khan", "email": "sara@example.com", "message": "Do you have vegan options?" }

Response  201 Created
{ "id": 8, "receivedAt": "2026-08-06T10:15:00Z" }
```

2.10 Error Format & Rate Limiting

All errors follow the same shape:

```
Response  400 Bad Request
{
  "statusCode": 400,
  "message": "Validation failed",
  "errors": ["price must be a positive number"]
}
```

Public write endpoints (POST /reservations, POST /contact) are rate-limited to 10 requests per 15 minutes per IP address, to prevent spam.

3. Docker & Deployment

3.1 Overview

The NestJS backend runs in Docker, alongside a PostgreSQL container, orchestrated with docker-compose. This keeps the local dev setup, staging, and production environments consistent.

3.2 Dockerfile (backend)

```
# ---- Build stage ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Production stage ----
FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

Multi-stage build — the final image only contains production dependencies and the compiled output, keeping it small.

3.3 docker-compose.yml

```
version: "3.9"

services:
  api:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://nobby:nobby_pass@db:5432/nobby_db
      - JWT_SECRET=${JWT_SECRET}
    depends_on:
      - db
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_USER=nobby
      - POSTGRES_PASSWORD=nobby_pass
      - POSTGRES_DB=nobby_db
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: unless-stopped

volumes:
  pgdata:
```

3.4 Environment Variables

Variable	Used By	Description
DATABASE_URL	API	PostgreSQL connection string
JWT_SECRET	API	Secret used to sign admin login tokens
PORT	API	Port the NestJS app listens on (default 3000)
NEXT_PUBLIC_API_URL	Frontend	Base URL the frontend calls for API requests
CDN_BASE_URL	API / Frontend	Base URL for media storage / CDN

3.5 Common Commands

```
# Build and start everything
docker compose up --build

# Run in the background
docker compose up -d

# View backend logs
docker compose logs -f api

# Stop everything
docker compose down

# Run a one-off database migration
docker compose exec api npm run migration:run
```

4. Frontend Technical Notes

4.1 Project Structure

```
frontend/
├── app/
│   ├── page.tsx               # Home page
│   ├── menu/page.tsx          # Menu showcase
│   ├── gallery/page.tsx       # Photo / video gallery
│   ├── reservations/page.tsx  # Reservation form
│   ├── contact/page.tsx       # Contact form
│   └── admin/                 # Admin dashboard routes
├── components/
├── lib/
│   └── api.ts                 # API client wrapper
├── public/
└── next.config.js
```

4.2 Data Fetching Strategy

Public pages (menu, gallery, testimonials) are statically generated and cached, then refreshed automatically using Incremental Static Regeneration (ISR) — so pages stay fast without needing a full rebuild for every content change.

When an admin saves a change (e.g. updates a menu item), the API triggers Next.js's on-demand revalidation endpoint, so the update appears on the live site within seconds instead of waiting for the next scheduled rebuild.

4.3 API Client Wrapper

```
// lib/api.ts
const API_URL = process.env.NEXT_PUBLIC_API_URL;

export async function getMenuItems(categoryId?: number) {
  const url = categoryId
    ? `${API_URL}/menu-items?category=${categoryId}`
    : `${API_URL}/menu-items`;

  const res = await fetch(url, { next: { revalidate: 60 } });
  if (!res.ok) throw new Error("Failed to load menu items");
  return res.json();
}
```

4.4 Image Handling

- All food and gallery photography is served through next/image for automatic resizing, lazy loading, and modern formats (WebP/AVIF).
- The CDN domain is whitelisted in next.config.js under images.domains so optimized images can be pulled from it directly.

5. Database Schema & SQL

5.1 Entity Relationships

- menu_items belongs to menu_categories (many items per category)
- gallery_images, testimonials, reservations, and contact_messages stand alone (no foreign keys to other tables)
- admin_users is used only for authentication, not linked to content tables

5.2 Table Definitions (DDL)

```
CREATE TABLE menu_categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    sort_order INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE menu_items (
    id SERIAL PRIMARY KEY,
    category_id INTEGER NOT NULL REFERENCES menu_categories(id) ON DELETE CASCADE,
    name VARCHAR(150) NOT NULL,
    description TEXT,
    price NUMERIC(10,2) NOT NULL,
    image_url TEXT,
    is_featured BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE gallery_images (
    id SERIAL PRIMARY KEY,
    image_url TEXT NOT NULL,
    caption VARCHAR(200),
    tag VARCHAR(50),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE testimonials (
    id SERIAL PRIMARY KEY,
    guest_name VARCHAR(100) NOT NULL,
    quote TEXT NOT NULL,
    rating SMALLINT CHECK (rating BETWEEN 1 AND 5),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE reservations (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    party_size SMALLINT NOT NULL,
    reservation_time TIMESTAMP NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE contact_messages (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL,
    message TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE admin_users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(150) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role VARCHAR(20) NOT NULL DEFAULT 'admin',
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX idx_menu_items_category ON menu_items(category_id);
CREATE INDEX idx_reservations_time ON reservations(reservation_time);
```

5.3 Example Queries

```
-- Get all menu items grouped by category, in display order
SELECT c.name AS category, i.name, i.description, i.price
FROM menu_items i
JOIN menu_categories c ON c.id = i.category_id
ORDER BY c.sort_order, i.name;

-- Get featured menu items for the homepage
SELECT name, price, image_url
FROM menu_items
WHERE is_featured = TRUE
ORDER BY created_at DESC
LIMIT 6;

-- Insert a new reservation
INSERT INTO reservations (name, phone, party_size, reservation_time)
VALUES ('Ali Raza', '+92 300 1234567', 4, '2026-08-10 20:00:00')
RETURNING id, status;

-- Get today's confirmed reservations
SELECT name, phone, party_size, reservation_time
FROM reservations
WHERE status = 'confirmed'
  AND reservation_time::date = CURRENT_DATE
ORDER BY reservation_time;

-- Average testimonial rating
SELECT ROUND(AVG(rating), 1) AS average_rating, COUNT(*) AS total_reviews
FROM testimonials;
```

5.4 Notes on Constraints & Indexes

- menu_items.category_id cascades on delete — removing a category removes its items, so this should require admin confirmation in the UI.
- testimonials.rating is constrained to 1–5 at the database level, as a safety net behind the API's own validation.
- Indexes on menu_items.category_id and reservations.reservation_time keep the two most frequent lookups (menu-by-category, reservations-by-date) fast as data grows.